Fast Gradient Boosting without Trees for Large-Scale Time Series Classification

Matthew Yang
Student ID 36196479

November 2, 2026

Abstract

A tree ensemble trained on a specialised feature transform, such as extremely randomised trees on QUANT features, is a canonical approach to time series classification, and is both accurate and cheap on standard benchmarks. Most of those benchmarks are small, however, and the memory and compute cost of training tree ensembles limits how far they scale. A boosted tree ensemble such as XGBoost searches every feature for split points in every tree of every boosting round, and keeps the whole training set in memory. We replace each tree with a *global partition*: a few feature splits, shared by all examples and each placed between two training examples of different classes, which together act as a hash whose bits index a table of leaf values. This removes the search for split points and the traversal of trees, since an example's leaf is read directly from its hash, and it removes the need to hold the training set in memory. The resulting booster, HashBoost, trains from mini-batches on the GPU, and its memory use does not grow with the size of the training set. On 26 MONSTER datasets with QUANT features, it matches tuned XGBoost's test accuracy and trains 8.2× faster in total on the same GPU, up to 31× on the largest datasets, though it is slower than XGBoost on the datasets that are cheapest to train. On the much smaller datasets of the UCR archive the advantage disappears, and QUANT's own ExtraTrees classifier is both faster and more accurate.

1 Introduction

Gradient boosted decision trees [1], in implementations such as XGBoost [2], LightGBM [3] and CatBoost [4], are the default choice for tabular prediction, where they remain competitive with, and frequently superior to, deep learning [5, 6]. Tree ensembles are also the usual classifier used with interval-based time series features: QUANT [7] passes its quantile features to extremely randomised trees, and PULSAR [8] features are classified the same way.

The MONSTER benchmark [9] has up to 5.9 million series per dataset, and a transform such as QUANT produces tens of thousands of features from each. At this scale, the memory requirements and computational cost of training traditional tree-based methods, such as extremely randomised trees or boosted trees, become prohibitive. To grow a boosted tree, the learner searches for split points on every feature, and a multiclass booster grows one tree per class in every round. The cost of a round is therefore proportional to the number of training examples, features and classes multiplied together. Histogram binning, approximate split finding, gradient-based sampling and feature bundling [2, 3, 10] all reduce the constant factor, but every round still has to revisit the whole training set, and the training set has to be held in memory.

In this paper we replace each tree with a *global partition* of the feature space: a small set of feature splits, shared by all examples, that can be treated like a hash function. A partition consists of b axis-aligned threshold predicates, that is, b splits of the form "feature j is at most t". Each example's b answers form a b-bit code, which indexes a table of 2^b leaf values. Each split takes a random feature and puts its threshold halfway between two *hard* examples with different

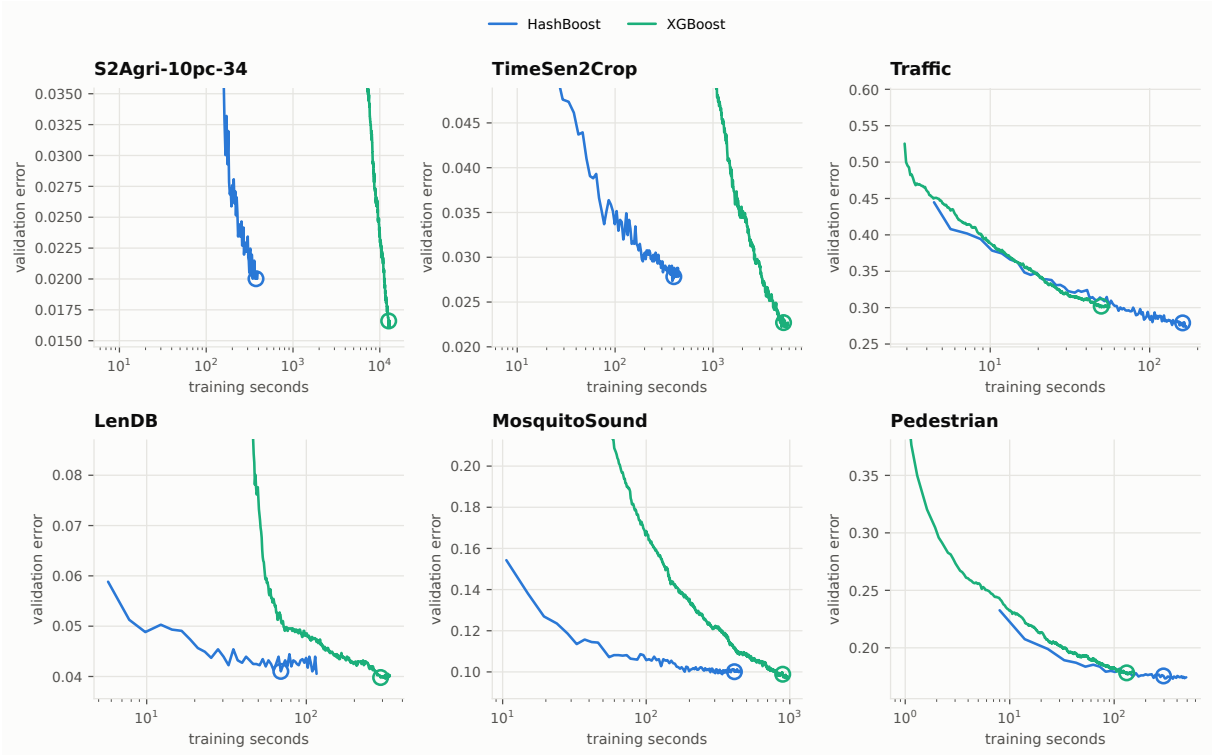


Figure 1: Validation error against training time for the six largest MONSTER training pools (fold 0, one L40S GPU, QUANT included). On S2Agri, TimeSen2Crop and LenDB, HashBoost reaches a low error in a few minutes, where XGBoost needs up to hours to reach the same error. Given its full budget, XGBoost finishes slightly lower on S2Agri and TimeSen2Crop. Each booster runs at its tuned configuration. HashBoost's time is measured at each evaluation, XGBoost's is its build time plus its mean time per round, and rings mark the point chosen on tune error. The time axis is logarithmic.

labels, meaning examples to which the current model gives a high training loss. Choosing a split therefore needs no search over the training set. A partition's splits are taken from the mini-batch in use when it is created, but they then apply unchanged to every later batch, so its leaf values can be re-estimated from all the data that follows, and the model never needs the whole training set in memory at once. Since nothing in a round depends on the size of the training set, HashBoost's cost per round stays flat as the training set grows, which is exactly where boosted trees become expensive (Figure 1).

In this work, we address the following three research questions:

RQ1 Given the same tuning effort, does HashBoost match a tuned boosted tree ensemble's accuracy on MONSTER's test folds?

RQ2 How do HashBoost's training time and memory compare with XGBoost's, and what explains the difference?

RQ3 Does HashBoost remain competitive in accuracy and speed on small training sets?

We compare HashBoost against XGBoost, tuned by the same procedure, and against QUANT's ExtraTrees, on 26 MONSTER datasets with training pools of up to 4.65 million examples, and on the 112 small datasets of the UCR archive. On MONSTER, HashBoost matches XGBoost's test accuracy while training 8.2 times faster in total on the same GPU, and its memory use does not grow with the training set. These advantages are limited to large datasets. On the datasets that are cheapest to train, XGBoost is faster, and on the UCR datasets ExtraTrees is both faster and more accurate than either booster.

The rest of this paper is structured as follows. Section 2 reviews gradient boosting, ways of scaling it, and tree-based time series classifiers. Section 3 describes HashBoost and its GPU

implementation, and Section 4 the datasets, baselines and tuning. Sections 5.1 to 5.3 answer RQ1 to RQ3 in turn, and Section 6 discusses the results and their limitations. Our code is available at `https://github.com/minip8/FIT2082`.

2 Background and Related Work

Boosting [11], and gradient boosting [1] in particular, builds an additive model one weak learner at a time, and fits each new learner to the errors that the current model still makes. Trees are the usual weak learner, but nothing in the framework requires them. XGBoost [2] sets each leaf's value with a second-order rule, a Newton step $-\sum g / \sum h$, where g and h are the gradient and hessian of the loss summed over the examples in that leaf. In other words, each leaf moves its examples' predictions in the direction that most reduces their loss, so each round corrects part of the error left by the rounds before it. Several boosters restrict the shape of the tree. CatBoost [4] grows *oblivious* trees, which use the same split at every node of a given depth, so a depth-d tree is d splits indexing 2^d leaves. Boosted decision tables [12] have the same structure. Like CatBoost, they choose each split greedily by gain over the whole training set, and they then *backfit* the splits, re-choosing each one given the others, but they set each table's leaves only once. Backfitting comes from additive models [13], where each component is refitted in turn while the others are held fixed.

Random forests [14] and extremely randomised trees [15] showed that, in a large enough ensemble, much of the split search can be replaced with randomness, which is the idea that HashBoost borrows for its splits. The usual way to scale tree boosting, though, is to keep the search and make it faster. GPU histogram construction [16] builds each tree's histograms in parallel, and out-of-core training [2] pages the binned training matrix in from disk, but both still pass over the whole training set in every round. Streaming methods avoid that pass. Hoeffding trees [17] grow a single tree from a stream, splitting a leaf once enough examples have arrived, and online boosting [18, 19] updates every weak learner as each example arrives. These methods are designed for data that arrives continuously, and are often less effective on large static datasets.

In time series classification, random-feature methods have shown that data-dependent search can often be replaced by cheap random transformations. ROCKET [20], MiniRocket [21] and Hydra [22] pair random convolutional kernels with a linear classifier, at a fraction of the cost of ensembles such as HIVE-COTE [23]. Interval methods (Time Series Forest [24], CIF [25], DrCIF [23], rSTSF [26], QUANT [7], PULSAR [8]) all end in a tree ensemble, so at scale they inherit its cost.

3 Method

HashBoost is a gradient booster whose weak learner is a global partition rather than a tree. It replaces only the classifier in the QUANT pipeline: QUANT turns each series into a fixed feature vector, and HashBoost trains on those features. Each weak learner's splits are chosen cheaply, from the current mini-batch alone, and then stay fixed, while its leaf values keep improving as more data passes through. A single random partition is a weak classifier, but a boosted sum of many, each aimed at the examples the model currently gets wrong, can be a strong one. This section defines a round (Section 3.1), how its splits are chosen (Section 3.2) and how its leaves are fitted (Section 3.3), then gives the cost of training (Section 3.4) and the implementation (Section 3.5).

3.1 A round is a global partition

Let $x \in \mathbb{R}^p$ be a feature vector and k the number of classes. A round r is defined by b pairs $(j_{r,i}, t_{r,i})$ of a feature index and a threshold, for $i = 0, \ldots, b-1$. It maps x to a b-bit code

$$c_r(x) = \sum_{i=0}^{b-1} 2^i \mathbb{1}\left[x_{j_{r,i}} \le t_{r,i}\right] \in \{0, \ldots, 2^b - 1\}, \tag{1}$$

and has a table $L_r \in \mathbb{R}^{2^b \times k}$ of leaf values. The model after R rounds is

$$F(x) = \sum_{r=1}^{R} L_r[c_r(x)] \in \mathbb{R}^k, \tag{2}$$

and predicts $\mathrm{softmax}(F(x))$. A round is therefore an oblivious tree of depth b whose splits are fixed when it is created. CatBoost's trees and boosted decision tables have the same shape (Section 2), but they choose each split by searching the whole training set, whereas HashBoost chooses its splits from one mini-batch (Section 3.2). By default $b = 8$, which gives each round 256 buckets.

Put simply, a round asks b yes-or-no questions of an example, each of the form "is feature j at most t?". The b answers form a binary number, which is the address of one row of the round's table, and that row holds the round's correction to the example's k class scores. The model's prediction is the sum of these corrections over all rounds. Since every example is asked the same questions, finding its leaf takes b comparisons and no tree traversal.

3.2 Choosing the splits

Given the current batch, its labels and the model's predicted probabilities, a new round's splits are chosen as follows:

1. Order the batch by cross entropy, largest first, so the examples the model gets most wrong come first.
2. Walk that order and greedily form b pairs of examples with different labels.
3. For each pair (u, v), draw a feature j uniformly at random and set $t = (u_j + v_j)/2$.

As in extremely randomised trees, each split is random, but it is placed between two examples that the model currently gets wrong, so it tends to separate examples that the model is confusing. The only search involved is the sort in the first step.

We also evaluate a *gain-selected* variant, because on the smaller pools the purely random choice of feature held HashBoost back (Section 6). The gain-selected variant draws a pool of bK hard-pair candidates and fills the bits one at a time, in the same way that CatBoost grows an oblivious tree level by level (Section 2). Each new split takes the unused candidate with the highest second-order gain, $\sum G^2/(H + \lambda)$ with $\lambda = 1$, summed over classes and over the cells made by the earlier splits, where G and H are the batch's gradient and hessian summed within a cell. Unlike CatBoost, HashBoost scores only these bK random candidates, on a single batch, instead of every feature over the whole training set.

3.3 Fitting the leaves

Following a standard approach in gradient boosting, we use the softmax cross-entropy gradient $g = p - \mathrm{onehot}(y)$ and a floored diagonal hessian $h = p(1-p) + 10^{-3}$. Every round keeps running sums $G_r, H_r \in \mathbb{R}^{2^b \times k}$ of $-g$ and h over the examples that landed in each bucket, and its leaf values are

$$L_r[s] = \eta \, \frac{G_r[s]}{H_r[s] + \epsilon}, \tag{3}$$

Algorithm 1 One HashBoost step on a mini-batch (X, y) of n examples, adding M rounds.

for $m = 1, \ldots, M$ **do**
  $F \leftarrow \sum_{r=1}^{R} L_r[c_r(X)]$ ▷ predict the batch: (n, k) logits
  $p \leftarrow \mathrm{softmax}(F)$ ▷ class probabilities
  $g \leftarrow p - \mathrm{onehot}(y); \quad h \leftarrow p(1-p) + 10^{-3}$ ▷ gradient and hessian
  choose the splits of round $R+1$ from hard pairs ▷ Section 3.2
  for $r = 1, \ldots, R+1$ **do** ▷ all rounds, including the new one
    $G_r[c_r(X)] \mathrel{+}= -g; \quad H_r[c_r(X)] \mathrel{+}= h$ ▷ add to bucket sums
    $L_r \leftarrow \eta\, G_r/(H_r + \epsilon)$ ▷ refresh leaf values
  end for
  $R \leftarrow R + 1$
end for

with learning rate $\eta = 0.1$. By default $\epsilon = 10^{-6}$. A larger ϵ acts as an L2 penalty that shrinks thinly populated leaves.

The main departure from tree boosting is *when* these sums are accumulated. In tree boosting, a tree's leaves are set once, when it is fitted, and never change. Boosted decision tables revisit their splits by backfitting, but they too set each table's leaves only once. HashBoost does the reverse: a round's splits are fixed, and its leaves keep being re-estimated. Every mini-batch is accumulated into every existing round, with gradients evaluated at the current model (Algorithm 1). Like online boosting, this updates every weak learner from new data, but since a round's structure is fixed, the update is a scatter-add rather than a refit. A round created early therefore keeps absorbing evidence from all the batches that follow, and is not tied to the batch that created it. New rounds can be fit one mini-batch at a time, while every round's leaves are still informed by the whole training set. Adding $M > 1$ rounds per batch raises capacity without needing more passes over the data.

3.4 What training costs

Consider a batch of n examples, a model of R rounds with b bits each, and k classes. The batch costs one encoding and one scatter-add for each round it touches, $O(nRb + nRk)$, plus the cost of choosing the new round's splits from that batch alone. None of this depends on the number N of examples in the training pool. The total cost is the number of batches times the cost per batch, and memory only has to hold one batch and the tables. A boosted tree's round, by contrast, builds feature histograms over all N examples for each of the k class trees. With p features, its cost per round grows with $N \cdot p \cdot k$, and it needs the binned matrix of all $N \times p$ values.

HashBoost does have a scaling problem of its own. Since every batch updates every round, a batch costs $O(R)$ once there are R rounds, and with a fixed number of rounds per batch, T batches cost $O(T^2)$. We address this through *round freezing*. With a window of W rounds, only the newest W keep accumulating, and once a round is frozen its contribution to an example never changes. If each training example has a fixed id, its frozen contribution can be summed once and cached in an (N, k) table. This cache is the one part of HashBoost's memory that grows with N, but at four bytes per example and class it is small next to the data (8 MB on LenDB, about 0.5 GB on S2Agri-10pc-34), and the model can run without this cache at $O(R)$ cost per batch. A batch then costs $O(W + f)$ round updates, where f is the number of rounds frozen since its examples were last seen, and the total cost becomes linear in the number of rounds.

3.5 Implementation

We implement HashBoost in PyTorch, and it runs entirely on the GPU. We also port the QUANT transform to PyTorch, so the whole pipeline, from raw series to predictions, stays on the GPU. This version is about 13× faster than our initial CPU implementation in numba, which we keep as an independent reference implementation. Both are in our repository.

The code is designed so that an experiment can change one component at a time. The split family, the split selector and the loss are separate interfaces (Python `Protocols`), so a new splitter or objective can be swapped in without touching the rest of the model. Every experimental change is added as an opt-in option, so the default model always computes the same algorithm as the reference implementation, and the reference can serve as a test oracle. A test suite of 224 tests replays the same hashes through both implementations and requires their predictions to agree within a tolerance, and it checks our GPU kernels and the QUANT port against their reference versions in the same way. Every run writes its configuration, data split and results to disk with the commit it ran from, so each number in this paper can be traced back to the code that produced it.

4 Experimental Setup

We evaluate HashBoost on 26 large datasets from MONSTER and on the 112 small datasets of the UCR archive. We compare it against XGBoost and ExtraTrees on test error, training time and memory.

MONSTER. We use 26 classification datasets from MONSTER [9], with QUANT [7] features (depth 6, div = 4, our PyTorch port). They range from 10,299 to 5.85 million series, from 2 to 82 classes, and from 212 to 117,746 QUANT features (Table 5 in Section A lists them all). MONSTER supplies five fixed test folds, and for each fold we call every series outside it the *pool*. Pools range from 7,898 to 4.66 million series. A fixed shuffle of the pool (seed 42) gives a validation slice and a tune slice of 4,096 rows each, or a fifth of the pool each when the pool has fewer than 73,728 rows. Every model trains on the rest. The tune slice is used for every choice made after training, such as early stopping, and validation error is used only to choose between configurations.

Configurations. We tuned HashBoost and XGBoost in the same way, on fold 0's validation error alone. We screened one change at a time on 10 of the 26 datasets (CrowdSourced, DREAMERA, FordChallenge, InsectSound, LenDB, PAMAP2, STEW, Traffic, UCIActivity and WISDM), kept two configurations per model, and ran both on all 26. For each dataset and model we froze the one with the lower fold-0 tune error, then trained it once on each of the five folds and scored it once on that fold's test series, at the round its tune error chose. No test series was seen before then.

Table 1 gives the two HashBoost configurations. Both train on batches of 4,096 rows for at most 5,000 batches, evaluate every 50, stop once their best tune evaluation is 1,000 batches old, and freeze rounds older than 128 batches. Over five model seeds on fold 0, the same configuration won every time on 25 of the 26 datasets.

Our main baseline is XGBoost, tuned with the same effort. We chose it because it trains on the same GPU, which makes the times directly comparable, and because it can train out of core (Section 5.2). Its screen first raised its round cap to 5,000, stopping early after 100 rounds without improvement, and then varied depth, learning rate, subsampling, L2 regularisation and the number of bins. The two final configurations are `d4` (depth 4, learning rate 0.1; picked on 17 datasets) and `d8` (depth 8, learning rate 0.05, 64 bins; picked on 9). We also compare against

Table 1: The two HashBoost configurations. `G` scores $K = 64$ candidates per bit, a pool of 384. Each dataset uses the configuration with the lower fold-0 tune error.

config	splits	bits	ϵ	rounds/batch	picked on
`G`	gain-selected	6	100	8	18 datasets
`L`	hard pairs	8	1	16	the largest pools (LenDB, both S2Agri, Traffic, Pedestrian), WhaleSounds, DREAMERA and DREAMERV

ExtraTrees with QUANT's own settings (200 trees, entropy criterion, a tenth of the features per split).

UCR. For smaller datasets, we use the 112 univariate UCR datasets [27] used by the bake-off [28]. These are the archive's 128 datasets, minus the 15 with missing values or unequal lengths, and minus Fungi, which has only one training series per class. We use each on its default train/test split, and the training sets range from 16 to 8,926 series (median 190). Since there is no validation split, every model runs at fixed settings and we report test error. HashBoost uses 800 rounds of 8 bits, ExtraTrees uses QUANT's settings, and XGBoost uses its library defaults (100 rounds, learning rate 0.3, depth 6).

Hardware and timing. Every HashBoost and XGBoost run on MONSTER had one NVIDIA L40S (46 GB) on Monash's M3 cluster to itself, so their times can be compared directly. ExtraTrees ran on CPU nodes with 36 cores. All reported times include the QUANT transform. The UCR runs and the streaming runs in Section 5.2 used one RTX 4060 (8 GB), with ExtraTrees on the same machine's 12-thread CPU. Seeding does not make HashBoost deterministic, because scatter-adds on the GPU accumulate in a nondeterministic order.

5 Results

5.1 Accuracy at scale (RQ1)

Table 2 gives each model's test error on the 26 MONSTER datasets, averaged over the five test folds, with training times, and Table 3 summarises the pairwise comparisons.

HashBoost and XGBoost come out level. HashBoost's error is 1.016 times XGBoost's in geometric mean, and the Wilcoxon test finds no difference (Table 3). A non-significant test does not by itself show that two models are equal, so we also estimate how large the gap could be. On a typical dataset, HashBoost's error is at most about 7% higher than XGBoost's, and it may be up to 3% lower (a 95% t-interval over the datasets' log error ratios puts the ratio between 0.97 and 1.07). On the few datasets where one model is clearly ahead, it is usually XGBoost, including both S2Agri subsets and TimeSen2Crop, which we return to in Section 6. HashBoost does relatively better on the 10 datasets used for tuning, where its error is about 3% below XGBoost's error, than on the other 16, where it is about 4.5% above XGBoost's error. Tuning on those 10 datasets therefore favoured HashBoost more than XGBoost.

Both boosters rank ahead of ExtraTrees. Ranking the three models on each dataset, both boosters average 1.77 and ExtraTrees 2.46, a gap just beyond the critical difference (Figure 2). For HashBoost the margin is narrow. Its pairwise Wilcoxon test against ExtraTrees is not significant, but the 95% interval on its error ratio, 0.84–0.97, lies entirely below 1: on a typical dataset HashBoost's error is between 3% and 16% lower than ExtraTrees'. ExtraTrees loses most on large acoustic and sensor pools such as MosquitoSound, AudioMNIST-DS and Pedestrian.

Table 2: MONSTER: test error, mean of the five test folds, one run per fold. HashBoost and XGBoost use the configuration each picked on fold 0. Times are training seconds summed over the five folds, QUANT included: HashBoost and XGBoost on one L40S, ExtraTrees on 36 CPU cores. The lowest error in each row is bold. †XGBoost's run on WISDM's test fold 3 failed (test error 0.683): that fold holds every series of two of the six classes, and XGBoost 3.4 estimates its intercept from a pool that lacks them. We report a rerun with the intercept fixed at 0.5, the default before version 2.0 (0.178), with the original run's time. Counting the failed run, HashBoost's error ratio to XGBoost is 0.992 ($p = 0.71$).

	HashBoost		XGBoost		ExtraTrees	time (s)		
dataset	config	error	config	error	error	HB	XGB	ET
AudioMNIST-DS	G	**0.0816**	d4	0.0884	0.1270	503	885	429
CornellWhaleChallenge	G	0.1250	d8	**0.1233**	0.1262	290	424	412
CrowdSourced	G	0.3501	d4	0.3622	**0.3396**	192	95	119
DREAMERA	L	0.5161	d8	0.5312	**0.4867**	438	1,775	4,253
DREAMERV	L	**0.4036**	d8	0.4038	0.4179	434	2,264	4,235
FordChallenge	G	0.0823	d8	0.0731	**0.0715**	196	152	224
FruitFlies	G	**0.0458**	d4	0.0462	0.0619	383	384	670
InsectSound	G	**0.1872**	d4	0.1891	0.2268	520	357	148
LakeIce	G	**0.0018**	d4	0.0019	0.0024	195	38	116
LenDB	L	0.0538	d4	**0.0522**	0.0551	728	1,364	25,843
MosquitoSound	G	0.1052	d4	**0.1008**	0.1596	1,855	5,054	7,366
Opportunity	G	0.1322	d4	**0.1305**	0.1394	399	852	761
PAMAP2	G	**0.2143**	d4	0.2246	0.2288	277	1,025	1,013
Pedestrian	L	**0.1746**	d8	0.1772	0.2135	2,588	792	61
S2Agri-10pc-17	L	0.2829	d8	0.2525	**0.2521**	1,463	30,068	18,291
S2Agri-10pc-34	L	0.2828	d8	**0.2530**	0.2638	2,013	61,933	16,151
STEW	G	**0.1937**	d4	0.1993	0.2371	325	310	382
Skoda	G	0.0567	d4	0.0610	**0.0565**	354	784	293
TimeSen2Crop	G	0.1412	d4	**0.0849**	0.1808	1,926	35,655	32,910
Tiselac	G	**0.2139**	d4	0.2140	0.2172	396	268	115
Traffic	L	0.2841	d8	0.2939	**0.2488**	846	407	375
UCIActivity	G	**0.0521**	d4	0.0675	0.0855	169	46	21
USCActivity	G	0.2428	d4	**0.2396**	0.2751	497	305	188
WISDM	G	0.1152	d4	**0.1131**†	0.1298	161	22	17
WISDM2	G	0.2556	d4	**0.2541**	0.2614	458	206	297
WhaleSounds	L	0.3128	d8	**0.2802**	0.2936	612	3,633	1,856
total						18,214	149,099	116,546

Table 3: MONSTER: pairwise comparisons over the 26 datasets. The ratio is the geometric mean of the first model's error over the second's. W/T/L counts the datasets where the first has the lower, equal and higher five-fold mean. "Clear" counts those where a 95% paired-t interval over the five per-fold differences excludes zero, either way. p is a two-sided Wilcoxon signed-rank test on the five-fold means, Holm-adjusted over the three comparisons.

comparison	ratio	W/T/L	clear (W/L)	p
HashBoost vs XGBoost	1.016	14/0/12	1/6	0.98
HashBoost vs ExtraTrees	0.904	18/0/8	9/5	0.11
XGBoost vs ExtraTrees	0.890	20/0/6	13/3	0.031

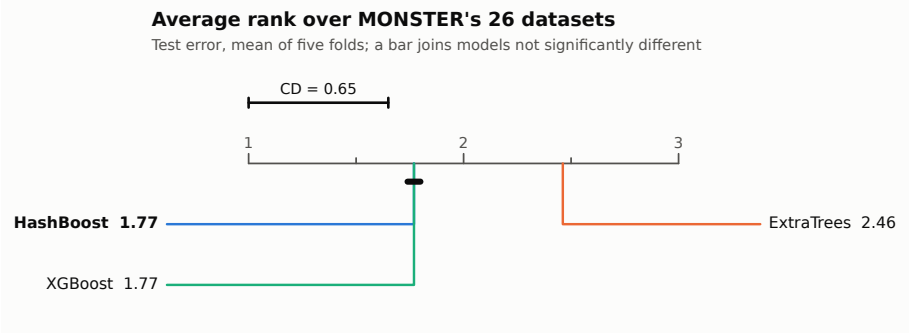


Figure 2: Critical difference diagram over MONSTER's 26 datasets [29]: test error, mean of the five folds. A Friedman test rejects equal ranks ($p = 0.016$). The bar joins models within the Nemenyi critical difference (0.65) of each other at $\alpha = 0.05$.

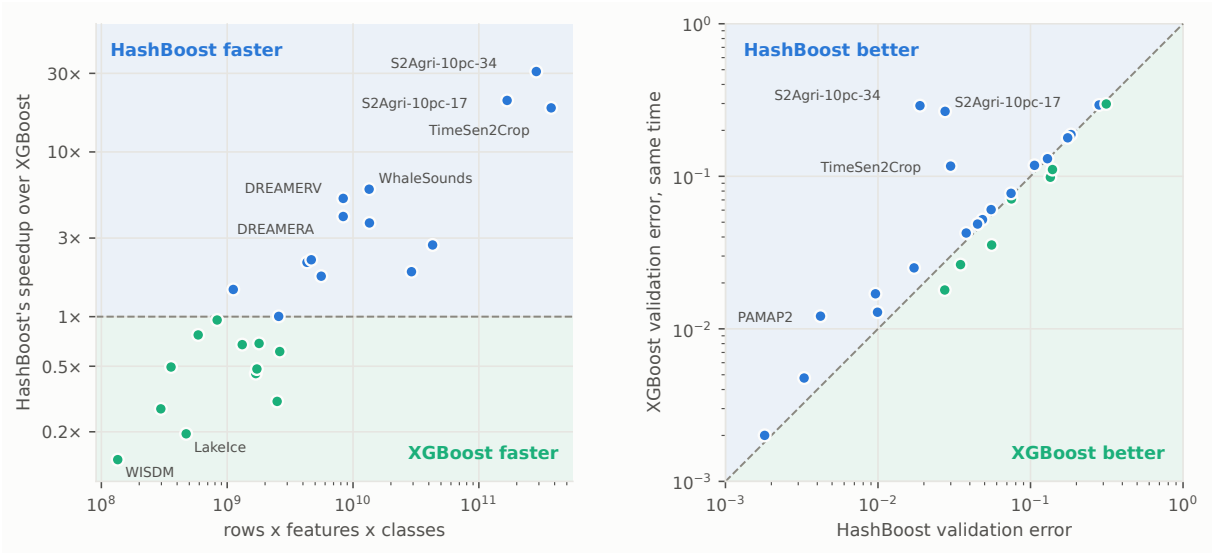


Figure 3: Left: HashBoost's training speedup over XGBoost on each MONSTER dataset (time summed over the five folds, one L40S) against training examples × features × classes. Right: validation error when XGBoost is given the same training time as HashBoost, mean of the five folds. XGBoost's round is chosen on tune error among the rounds it completed in that time. In both panels, blue regions and points mark datasets where HashBoost wins, and green where XGBoost wins.

5.2 Where the time goes (RQ2)

Training time. On the same GPU, HashBoost trained 8.2 times faster than XGBoost in total (Table 2), but that total hides a wide range. HashBoost is faster on 14 of the 26 datasets, ranging from 31 times faster on S2Agri-10pc-34 to 7 times slower on WISDM. Figure 3 (left) shows what decides this. The speedup rises with $N \cdot p \cdot k$, the product of training examples, features and classes that sets XGBoost's cost per round (Spearman $\rho = 0.87$), whereas HashBoost's cost is set by its batch budget. On the cheapest datasets, such as WISDM, LakeIce and UCIActivity, XGBoost finishes in seconds, and HashBoost's fixed cost per round makes it the slower of the two. On the most expensive ones, S2Agri and TimeSen2Crop, XGBoost takes hours while HashBoost takes minutes.

Equal training time. We also compare the two boosters when XGBoost is limited to HashBoost's training time. In Figure 3 (right), XGBoost gets exactly HashBoost's total training time on each fold, including the batches HashBoost ran after its best round before early stopping ended it, and its round is chosen on tune error from the rounds it completed in that time. Under this limit HashBoost is ahead on validation error on 19 of the 26 datasets. Across all

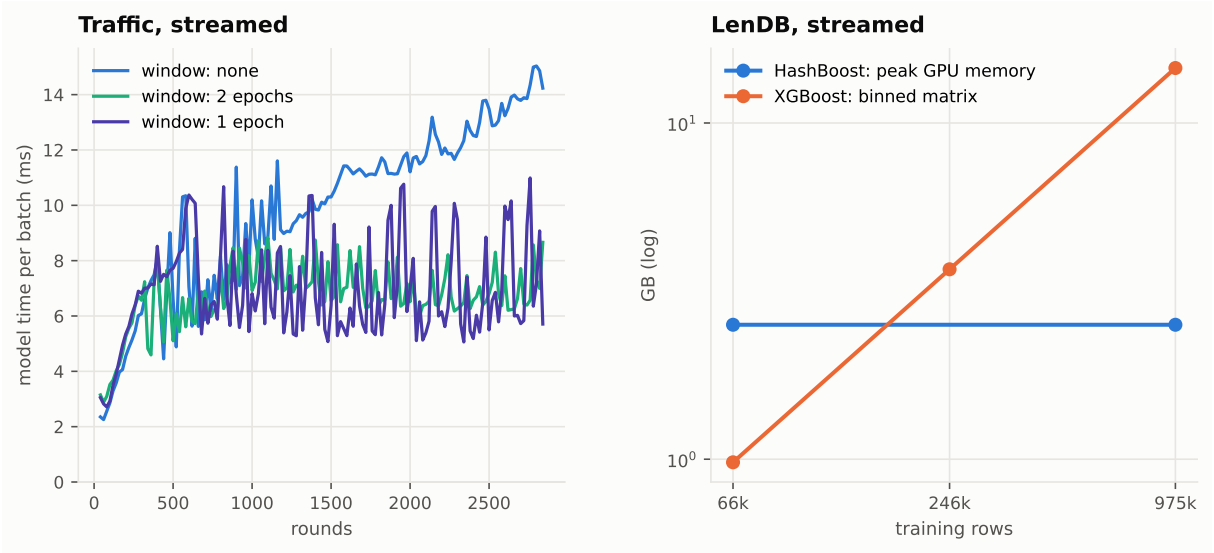


Figure 4: Left: HashBoost's model time per batch against rounds on a whole-pool Traffic stream (1.16 million rows, one round per batch, RTX 4060), unfrozen and with rounds older than one or two epochs frozen. Right: what each model needs available as LenDB's training pool grows. HashBoost's peak GPU memory is measured; XGBoost's binned matrix is measured on disk at 245,760 and 975,291 rows and derived from the same bytes per element at 65,536.

26 the difference is not significant (Wilcoxon $p = 0.24$), because on the cheaper datasets XGBoost finishes within HashBoost's time anyway. Where training is expensive, however, the gap is large. Given HashBoost's time, XGBoost only reaches a validation error of 0.27–0.29 on the two S2Agri subsets, about ten times HashBoost's 0.02–0.03, and the gap on TimeSen2Crop is similar. Figure 1, in the introduction, shows the curves for the six largest pools.

Memory. HashBoost's memory does not grow with the training set. To show this, we streamed LenDB's whole training pool (975,291 rows, which is 6.3 GB as raw series and 58 GB after QUANT) through both models on the RTX 4060. HashBoost's peak GPU memory was 2,572 MB both at 65,536 rows and at the full pool, because each batch is folded into the tables and then freed. XGBoost's binned matrix grows linearly with the pool, reaching 14.6 GB (Figure 4, right). On the 8 GB card, XGBoost had to page that matrix in from disk every round, which took 116 s per round, compared with 0.68 s at 65,536 rows. In the MONSTER benchmark on the L40S, tuned XGBoost used about 17 GB of GPU memory on LenDB, while HashBoost used at most 4.9 GB.

Round freezing. Without freezing, the cost per batch grows with the number of rounds. On a streamed Traffic run it rose from 2 ms to 14.5 ms over 2,840 rounds, whereas a window of one or two epochs' worth of rounds kept it level (Figure 4, left). On Pedestrian, with two rounds per batch, a window of ten epochs matched the unfrozen model's validation error in 2.4 times less time.

5.3 The small-data boundary (RQ3)

The UCR 112 training sets are small, and on them HashBoost loses both its accuracy and its speed advantage (Table 4). Our ExtraTrees reproduces the published QUANT results [28]. It is the most accurate model, significantly ahead of both boosters (Figure 5), and also the fastest, taking under half of HashBoost's time. At these sizes, most of a round's cost is fixed overhead rather than arithmetic. With its MONSTER defaults, HashBoost is level with default XGBoost. Its 8-bit rounds were set for buckets that see thousands of examples, and most UCR training

Table 4: UCR 112: test error on the default splits, one run per dataset. Mean rank is among the three models we ran. W/T/L is HashBoost's record against each row. p is a two-sided Wilcoxon signed-rank test against HashBoost, Holm-adjusted over the four comparisons. The published rows are the bake-off's resample 0, which is the default split.

model	mean error	median	mean rank	W/T/L	p	total fit
HashBoost	0.1941	0.1705	2.36	–	–	183 s
ExtraTrees	**0.1475**	0.1205	**1.25**	8/8/96	2.6×10^{-14}	70 s
XGBoost	0.2101	0.1912	2.39	58/3/51	0.082	413 s
QUANT (published)	0.1456	0.1177		13/5/94	4.9×10^{-14}	
HC2 (published)	0.1236	0.0753		8/7/97	3.8×10^{-15}	

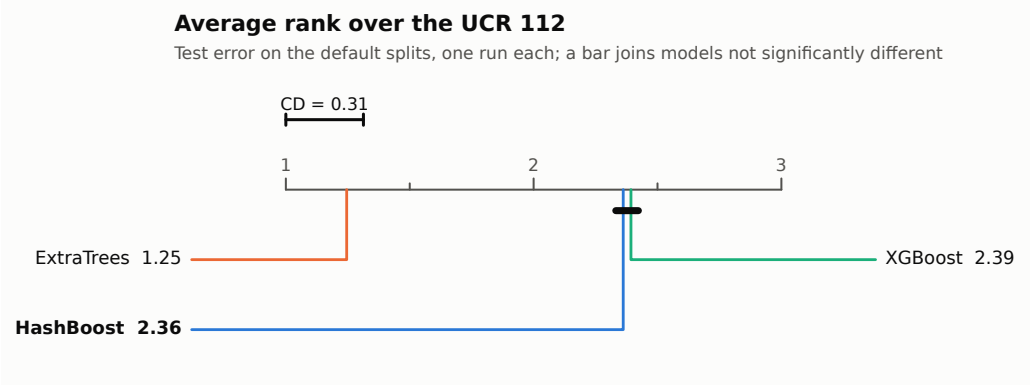


Figure 5: Critical difference diagram over the UCR 112 [29]. A Friedman test rejects equal ranks ($p = 3.6 \times 10^{-22}$). A bar joins models within the Nemenyi critical difference (0.31) of each other at $\alpha = 0.05$.

sets are too small to fill them. When we resize it for small data, using two bits and 3,200 rounds chosen by two-fold cross-validation over the datasets, HashBoost is significantly ahead of XGBoost (lower error on 82 datasets against 29), but still behind ExtraTrees (28 against 63), and it takes 7.5 times its default training time. So the best classifier for QUANT's features depends on scale. An average of random trees works best on small training sets, and a booster works best on large ones.

6 Discussion

The datasets where HashBoost saves the most time are also the ones where it loses the most accuracy to a fully trained XGBoost. Because the two models' costs scale differently, HashBoost is fastest where the classifier's cost is a problem in the first place, which is where it is meant to be used. But on those datasets, once XGBoost has run to completion, it overtakes HashBoost. HashBoost's test error is 12% higher than XGBoost's on S2Agri and 66% higher on TimeSen2Crop, even though HashBoost is far ahead at equal time. On S2Agri at least part of that gap comes from the budget rather than from the model. On 14 of the 15 S2Agri and Traffic folds, HashBoost hit its 5,000-batch cap with its best tune error in the last 1,000 batches, so it was still improving when it stopped, and that budget had been tuned on smaller pools. TimeSen2Crop is harder to explain away. There HashBoost hit the cap on only two of the five folds, and the gap is the largest in the benchmark.

Tuning also showed us what HashBoost needs. On the smaller pools the random choice of feature for each split held it back, and scoring candidates by gain was the biggest single improvement we found. Gain selection overfits the largest pools, however, because each split is the best of hundreds of candidates on one 4,096-row batch and so can fit that batch's noise. That is why two configurations survived (Table 1): gain-selected rounds for most datasets, and

plain hard pairs for the largest pools and a few others. We also tested whether short freezing windows would regularise the model. They did not, so freezing is useful only for speed.

Limitations. The evidence is limited in three ways. First, each result comes from a single run. MONSTER has one run per test fold, and the UCR results are one run on the default split, where the bake-off averages 30 resamples. Over five model seeds on fold 0, HashBoost's standard deviation was 0.0002 to 0.0024 on most datasets, so a rerun would move its error by at most about a quarter of a percentage point. Second, the tuning was narrow. Both models were tuned for accuracy rather than speed, and only on 10 of the 26 datasets, which favour HashBoost (Section 5.1). Validation error was also optimistic: for all three models, test error is about 3.3 times validation error, because the test folds are distributed differently from the pool that the validation and tune slices are drawn from. Every model was tuned on the same slices, so the comparison stays fair, but configurations chosen on validation error may not be the best on test. The time-matched comparison also uses validation error, since test error was scored only at each model's chosen round, and XGBoost's time there is estimated as its build time plus its mean time per round. Third, we compared against XGBoost and ExtraTrees only, not LightGBM or CatBoost.

Several directions follow. A round budget that grows with the training pool should buy accuracy on the largest datasets for a few more minutes of training, and a configuration tuned for speed rather than accuracy would show the trade-off between the two on the test folds. On small data, choosing the bit width from the number of examples per class might let one configuration serve both UCR and MONSTER, and cheaper rounds would cut HashBoost's fixed overhead.

7 Conclusion

HashBoost shows that gradient boosting can match tuned XGBoost at scale without growing trees or searching the training set. It suits large training pools, where XGBoost's cost per round is the bottleneck, and it is the wrong choice on small datasets, where ExtraTrees is both faster and more accurate. The open question is how much of the remaining gap on the very largest datasets a budget that grows with the pool can close.

Acknowledgements

We thank Angus Dempster for supervising this project. This work was supported by the MASSIVE HPC facility (`www.massive.org.au`).

References

[1] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, pp. 3146–3154, 2017.

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems 31*, pp. 6638–6648, 2018.

[5] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.

[6] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems 35: Datasets and Benchmarks Track*, pp. 507–520, 2022.

[7] A. Dempster, D. F. Schmidt, and G. I. Webb, "QUANT: A minimalist interval method for time series classification," *Data Mining and Knowledge Discovery*, vol. 38, no. 4, pp. 2377–2402, 2024.

[8] N. Cabello and L. Kulik, "PULSAR: Advancing interval-based time series classification to state-of-the-art performance," in *2025 IEEE International Conference on Data Mining (ICDM)*, pp. 1085–1094, 2025.

[9] A. Dempster, N. M. Foumani, C. W. Tan, L. Miller, A. Mishra, M. Salehi, C. Pelletier, D. F. Schmidt, and G. I. Webb, "MONSTER: Monash scalable time series evaluation repository." arXiv:2502.15122, 2025.

[10] J. H. Friedman, "Stochastic gradient boosting," *Computational Statistics & Data Analysis*, vol. 38, no. 4, pp. 367–378, 2002.

[11] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[12] Y. Lou and M. Obukhov, "BDT: Gradient boosted decision tables for high accuracy and scoring efficiency," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 1893–1901, 2017.

[13] A. Buja, T. Hastie, and R. Tibshirani, "Linear smoothers and additive models," *The Annals of Statistics*, vol. 17, no. 2, pp. 453–510, 1989.

[14] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[15] P. Geurts, D. Ernst, and L. Wehenkel, "Extremely randomized trees," *Machine Learning*, vol. 63, no. 1, pp. 3–42, 2006.

[16] R. Mitchell and E. Frank, "Accelerating the XGBoost algorithm using GPU computing," *PeerJ Computer Science*, vol. 3, p. e127, 2017.

[17] P. Domingos and G. Hulten, "Mining high-speed data streams," in *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 71–80, 2000.

[18] N. C. Oza and S. Russell, "Online bagging and boosting," in *Proceedings of the Eighth International Workshop on Artificial Intelligence and Statistics*, pp. 229–236, 2001.

[19] A. Beygelzimer, E. Hazan, S. Kale, and H. Luo, "Online gradient boosting," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[20] A. Dempster, F. Petitjean, and G. I. Webb, "ROCKET: Exceptionally fast and accurate time series classification using random convolutional kernels," *Data Mining and Knowledge Discovery*, vol. 34, no. 5, pp. 1454–1495, 2020.

[21] A. Dempster, D. F. Schmidt, and G. I. Webb, "MiniRocket: A very fast (almost) deterministic transform for time series classification," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, (New York), pp. 248–257, ACM, 2021.

[22] A. Dempster, D. F. Schmidt, and G. I. Webb, "Hydra: Competing convolutional kernels for fast and accurate time series classification," *Data Mining and Knowledge Discovery*, vol. 37, no. 5, pp. 1779–1805, 2023.

[23] M. Middlehurst, J. Large, M. Flynn, J. Lines, A. Bostrom, and A. Bagnall, "HIVE-COTE 2.0: A new meta ensemble for time series classification," *Machine Learning*, vol. 110, no. 11–12, pp. 3211–3243, 2021.

[24] H. Deng, G. Runger, E. Tuv, and M. Vladimir, "A time series forest for classification and feature extraction," *Information Sciences*, vol. 239, pp. 142–153, 2013.

[25] M. Middlehurst, J. Large, and A. Bagnall, "The canonical interval forest (CIF) classifier for time series classification," in *2020 IEEE International Conference on Big Data (Big Data)*, pp. 188–195, 2020.

[26] N. Cabello, E. Naghizade, J. Qi, and L. Kulik, "Fast, accurate and explainable time series classification through randomization," *Data Mining and Knowledge Discovery*, vol. 38, no. 2, pp. 748–811, 2024.

[27] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The UCR time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

[28] M. Middlehurst, P. Schäfer, and A. Bagnall, "Bake off redux: A review and experimental evaluation of recent time series classification algorithms," *Data Mining and Knowledge Discovery*, vol. 38, no. 4, pp. 1958–2031, 2024.

[29] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

A The MONSTER datasets

Table 5 lists the 26 MONSTER datasets used in Section 4, with the size of each training pool on test fold 0.

Table 5: The 26 MONSTER datasets. "Pool" is the number of series outside test fold 0, and "train" the rows trained on after the validation and tune slices are taken from it. Classes are those present in the labels.

dataset	series	pool	train	classes	QUANT features
AudioMNIST-DS	30,000	24,000	15,808	10	35,349
CornellWhaleChallenge	30,000	24,000	15,808	2	35,349
CrowdSourced	12,289	9,456	5,674	2	31,542
DREAMERA	170,246	140,638	132,446	2	31,542
DREAMERV	170,246	140,638	132,446	2	31,542
FordChallenge	36,257	29,003	20,811	2	14,130
FruitFlies	34,518	27,614	19,422	3	44,031
InsectSound	50,000	40,000	32,768	10	5,470
LakeIce	129,280	103,424	95,232	3	1,652
LenDB	1,244,942	983,483	975,291	2	14,940
MosquitoSound	279,566	223,652	215,460	6	33,178
Opportunity	17,386	12,184	7,312	5	117,746
PAMAP2	38,856	28,961	20,769	12	54,184
Pedestrian	189,621	151,696	143,504	82	212
S2Agri-10pc-17	5,850,881	4,661,858	4,653,666	17	2,120
S2Agri-10pc-34	5,850,881	4,661,858	4,653,666	29	2,120
STEW	28,512	21,384	13,192	2	31,542
Skoda	14,117	11,290	6,774	11	62,520
TimeSen2Crop	1,135,511	839,971	831,779	15	30,096
Tiselac	99,687	79,907	71,715	9	2,040
Traffic	1,460,968	1,168,774	1,160,582	7	212
UCIActivity	10,299	7,898	4,740	6	10,440
USCActivity	56,228	43,060	34,868	12	6,252
WISDM	17,166	11,989	7,195	6	3,126
WISDM2	149,034	98,094	89,902	6	3,126
WhaleSounds	105,163	84,130	75,938	8	22,105